

F# List, Set, Map, and Seq Library Reference

Generated reference for FSharp.Core collection modules: List, Set, Map, and Seq. The APIs are in namespace FSharp.Collections and assembly FSharp.Core.dll. F# collection modules use immutable collections: operations that appear to modify a collection generally return a new collection rather than changing the original.

Sources: FSharp.Core API reference pages for List, Seq, Map, and Set; Microsoft Learn F# collection type guidance. The source pages state that List/Seq/Map/Set modules contain operations for their respective collection types, and Microsoft Learn describes the standard F# collection types as functional/immutable except arrays.

Conventions: "predicate" means a function returning bool. "projection" maps an element to a key or numeric value. "chooser" maps an input to Some result or None. Functions named try... return option instead of raising when no result exists. Seq functions are generally lazy: many return a sequence that does work when enumerated.

Higher-order function examples

These functions accept other functions as input. The same shape appears in List, Seq, Set, and Map where applicable.

```
let numbers = [1; 2; 3; 4]

numbers |> List.map (fun x -> x * x)           // [1; 4; 9; 16]
numbers |> List.filter (fun x -> x % 2 = 0) // [2; 4]
numbers |> List.fold (fun acc x -> acc + x) 0 // 10
numbers |> List.choose (fun x -> if x % 2 = 0 then Some(string x) else None) // ["2"; "4"]

Map ["a", 1; "b", 2] |> Map.map (fun key value -> key + string value)
set [1;2;3] |> Set.exists (fun x -> x > 2)
seq { 1..100 } |> Seq.map (fun x -> x * x) |> Seq.take 5
```

List Module

List works with immutable, ordered, singly-linked lists. It is best for recursive processing and efficient head/tail access; appending/indexing require traversal.

List.allPairs

Usage	List.allPairs list1 list2
Inputs	list1: first list; list2: second list
Output	list of tuples containing every pairing
Description	Creates the Cartesian product of two lists, preserving nested-loop order: each element of list1 is paired with each element of list2.

```
List.allPairs [1;2] ["a";"b"] // [(1,"a"); (1,"b"); (2,"a"); (2,"b")]
```

List.append

Usage	List.append list1 list2
Inputs	two lists of the same element type
Output	one list
Description	Returns a new list containing list1 followed by list2. The original lists are unchanged.

```
List.append [1;2] [3;4] // [1;2;3;4]
```

List.average

Usage	List.average list
Inputs	numeric list
Output	average value
Description	Computes the arithmetic mean. Fails for an empty list.

```
List.average [1.0; 2.0; 3.0] // 2.0
```

List.averageBy

Usage	List.averageBy projection list
Inputs	projection: element -> numeric value; list
Output	average of projected values
Description	Maps each element to a numeric value and averages those values. Higher-order function.

```
["aa"; "bbbb"] |> List.averageBy (fun s -> float s.Length) // 3.0
```

List.choose

Usage	List.choose chooser list
Inputs	chooser: element -> option; list
Output	list of unwrapped Some results
Description	Combines filtering and mapping: the chooser returns Some value to keep or None to drop. Higher-order function.

```
["1"; "x"; "2"] |> List.choose (fun s -> match System.Int32.TryParse s with true,v -> Some v | _ -> None) // [1;2]
```

List.chunkBySize

Usage	List.chunkBySize size list
Inputs	size: positive chunk size; list
Output	list of lists
Description	Splits the list into chunks of at most size elements.

```
[1..7] |> List.chunkBySize 3 // [[1;2;3]; [4;5;6]; [7]]
```

List.collect

Usage	List.collect mapping list
Inputs	mapping: element -> list; list
Output	concatenated list
Description	Maps each element to a list and concatenates the results. Also called flatMap/bind. Higher-order function.

```
[1;2;3] |> List.collect (fun x -> [x; x*x]) // [1;1;2;4;3;9]
```

List.compareWith

Usage	List.compareWith comparer list1 list2
Inputs	comparer: elt -> elt -> int; two lists
Output	negative, zero, or positive int
Description	Lexicographically compares two lists using the supplied element comparison. Higher-order function.

```
List.compareWith compare [1;2] [1;3] // negative
```

List.concat

Usage	List.concat lists
Inputs	sequence/list of lists

Output	single flattened list
Description	Concatenates all inner lists in order.

```
List.concat [[1;2]; [3]; []] // [1;2;3]
```

List.contains

Usage	List.contains value list
Inputs	value to test; list
Output	bool
Description	Returns true when any element equals the value.

```
[1;2;3] |> List.contains 2 // true
```

List.countBy

Usage	List.countBy projection list
Inputs	projection: element -> key; list
Output	list of key * count pairs
Description	Groups by a projected key and counts elements in each group. Higher-order function.

```
["a";"bb";"c"] |> List.countBy String.length // [(1,2); (2,1)]
```

List.distinct

Usage	List.distinct list
Inputs	list
Output	list with duplicates removed
Description	Removes duplicate elements, keeping first occurrences.

```
[1;2;1;3;2] |> List.distinct // [1;2;3]
```

List.distinctBy

Usage	List.distinctBy projection list
Inputs	projection: element -> key; list
Output	list
Description	Removes duplicates based on projected keys, keeping first occurrence per key. Higher-order function.

```
["AL";"bob";"Amy"] |> List.distinctBy (fun s -> s.[0]) // ["AL"; "bob"]
```

List.empty

Usage	List.empty
Inputs	no input
Output	empty list
Description	The empty list value for any element type.

```
let xs : int list = List.empty
```

List.exactlyOne

Usage	List.exactlyOne list
Inputs	list
Output	single element
Description	Returns the only element. Raises if the list is empty or has more than one element.

```
List.exactlyOne [42] // 42
```

List.except

Usage	List.except exclusions list
Inputs	exclusions: seq of values; list
Output	filtered list
Description	Returns elements whose values are not in exclusions.

```
[1;2;3;4] |> List.except [2;4] // [1;3]
```

List.exists

Usage	List.exists predicate list
Inputs	predicate: element -> bool; list
Output	bool
Description	Returns true if at least one element satisfies the predicate. Short-circuits. Higher-order function.

```
[1;2;3] |> List.exists (fun x -> x > 2) // true
```

List.exists2

Usage	List.exists2 predicate list1 list2
Inputs	predicate: elt1 -> elt2 -> bool; two equal-length lists
Output	bool
Description	Tests paired elements and returns true if any pair satisfies the predicate. Higher-order function.

```
List.exists2 (fun a b -> a+b=10) [1;4] [9;2] // true
```

List.filter

Usage	List.filter predicate list
Inputs	predicate: element -> bool; list
Output	filtered list
Description	Keeps only elements for which the predicate returns true. Higher-order function.

```
[1..6] |> List.filter (fun x -> x % 2 = 0) // [2;4;6]
```

List.find

Usage	List.find predicate list
Inputs	predicate; list
Output	first matching element
Description	Returns the first element satisfying predicate. Raises if none match.

```
[1..5] |> List.find (fun x -> x > 3) // 4
```

List.findBack

Usage	List.findBack predicate list
Inputs	predicate; list
Output	last matching element
Description	Searches from the end and returns the last element satisfying predicate. Raises if none.

```
[1..5] |> List.findBack (fun x -> x < 5) // 4
```

List.findIndex

Usage	List.findIndex predicate list
Inputs	predicate; list
Output	zero-based index
Description	Returns the index of the first matching element. Raises if none.

```
["a";"bb"] |> List.findIndex (fun s -> s.Length=2) // 1
```

List.findIndexBack

Usage	List.findIndexBack predicate list
Inputs	predicate; list
Output	zero-based index
Description	Returns the index of the last matching element. Raises if none.

```
[1;2;3;2] |> List.findIndexBack ((=) 2) // 3
```

List.fold

Usage	List.fold folder state list
Inputs	folder: state -> element -> state; initial state; list
Output	final state
Description	Traverses left to right, threading an accumulator. Higher-order function.

```
[1;2;3] |> List.fold (fun acc x -> acc + x) 0 // 6
```

List.fold2

Usage	List.fold2 folder state list1 list2
Inputs	folder: state -> elt1 -> elt2 -> state; state; two equal-length lists
Output	final state
Description	Folds over paired elements from two lists. Higher-order function.

```
List.fold2 (fun acc a b -> acc + a*b) 0 [1;2] [10;20] // 50
```

List.foldBack

Usage	List.foldBack folder list state
Inputs	folder: element -> state -> state; list; state
Output	final state
Description	Traverses right to left, threading an accumulator. Higher-order function.

```
List.foldBack (fun x acc -> x::acc) [1;2] [] // [1;2]
```

List.foldBack2

Usage	List.foldBack2 folder list1 list2 state
Inputs	folder: elt1 -> elt2 -> state -> state; lists; state
Output	final state
Description	Right-to-left fold over paired elements from two lists. Higher-order function.

```
List.foldBack2 (fun a b acc -> (a,b)::acc) [1;2] [3;4] [] // [(1,3);(2,4)]
```

List.forall

Usage	List.forall predicate list
Inputs	predicate; list
Output	bool
Description	Returns true if every element satisfies predicate. Short-circuits.

```
[2;4;6] |> List.forall (fun x -> x % 2 = 0) // true
```

List.forall2

Usage	List.forall2 predicate list1 list2
Inputs	predicate over pairs; two equal-length lists
Output	bool
Description	Returns true if every pair satisfies predicate.

```
List.forall2 (=) [1;2] [1;2] // true
```

List.groupBy

Usage	List.groupBy projection list
Inputs	projection: element -> key; list
Output	list of key * group list
Description	Groups elements by a projected key. Higher-order function.

```
["a";"bb";"c"] |> List.groupBy String.length // [(1,["a";"c"]); (2,["bb"])]
```

List.head

Usage	List.head list
Inputs	non-empty list
Output	first element
Description	Returns first element. Raises for empty list.

```
List.head [10;20] // 10
```

List.indexed

Usage	List.indexed list
Inputs	list
Output	list of index * element
Description	Pairs each element with its zero-based index.

```
List.indexed ["a";"b"] // [(0,"a");(1,"b")]
```

List.init

Usage	List.init count initializer
Inputs	count; initializer: index -> element
Output	new list
Description	Creates a list by calling initializer for indexes 0..count-1. Higher-order function.

```
List.init 4 (fun i -> i*i) // [0;1;4;9]
```

List.insertAt

Usage	List.insertAt index value list
Inputs	index; value; list
Output	new list
Description	Inserts a value at the specified index.

```
[1;3] |> List.insertAt 1 2 // [1;2;3]
```

List.insertManyAt

Usage	List.insertManyAt index values list
Inputs	index; values seq/list; list
Output	new list
Description	Inserts several values at the specified index.

```
[1;4] |> List.insertManyAt 1 [2;3] // [1;2;3;4]
```

List.isEmpty

Usage	List.isEmpty list
Inputs	list
Output	bool
Description	Returns true if the list has no elements.

```
List.isEmpty [] // true
```

List.item

Usage	List.item index list
Inputs	zero-based index; list

Output	element
Description	Returns the element at index. Raises if out of range.

```
["a";"b"] |> List.item 1 // "b"
```

List.iter

Usage	List.iter action list
Inputs	action: element -> unit; list
Output	unit
Description	Applies an action to each element for side effects. Higher-order function.

```
[1;2] |> List.iter (printfn "%d")
```

List.iter2

Usage	List.iter2 action list1 list2
Inputs	action over pairs; equal-length lists
Output	unit
Description	Applies an action to paired elements.

```
List.iter2 (fun a b -> printfn "%d:%s" a b) [1;2] ["a";"b"]
```

List.iteri

Usage	List.iteri action list
Inputs	action: index -> element -> unit; list
Output	unit
Description	Applies an indexed action to each element.

```
["a";"b"] |> List.iteri (fun i x -> printfn "%d=%s" i x)
```

List.iteri2

Usage	List.iteri2 action list1 list2
Inputs	action: index -> elt1 -> elt2 -> unit; lists
Output	unit
Description	Applies an indexed action to paired elements.

```
List.iteri2 (fun i a b -> printfn "%d %A %A" i a b) [1] [2]
```

List.last

Usage	List.last list
Inputs	non-empty list
Output	last element
Description	Returns last element. Raises for empty list.

```
List.last [1;2;3] // 3
```

List.length

Usage	List.length list
Inputs	list
Output	int
Description	Returns number of elements.

```
List.length [1;2;3] // 3
```

List.map

Usage	List.map mapping list
Inputs	mapping: element -> result; list
Output	mapped list
Description	Transforms each element. Higher-order function.

```
[1;2;3] |> List.map (fun x -> x*x) // [1;4;9]
```

List.map2

Usage	List.map2 mapping list1 list2
Inputs	mapping: elt1 -> elt2 -> result; equal-length lists
Output	mapped list
Description	Transforms paired elements from two lists.

```
List.map2 (+) [1;2] [10;20] // [11;22]
```

List.map3

Usage	List.map3 mapping list1 list2 list3
Inputs	mapping over triples; equal-length lists
Output	mapped list
Description	Transforms corresponding triples from three lists.

```
List.map3 (fun a b c -> a+b+c) [1] [2] [3] // [6]
```

List.map1

Usage	List.map1 mapping list
Inputs	mapping: index -> element -> result; list
Output	mapped list
Description	Indexed map. Higher-order function.

```
["a";"b"] |> List.map1 (fun i x -> string i + x) // ["0a";"1b"]
```

List.mapi2

Usage	List.mapi2 mapping list1 list2
Inputs	mapping: index -> elt1 -> elt2 -> result; lists
Output	mapped list
Description	Indexed map over pairs.

```
List.mapi2 (fun i a b -> i+a+b) [1] [2] // [3]
```

List.max

Usage	List.max list
Inputs	non-empty comparable list
Output	maximum element
Description	Returns greatest element using generic comparison.

```
List.max [3;1;2] // 3
```

List.maxBy

Usage	List.maxBy projection list
Inputs	projection; non-empty list
Output	element with maximum projected key
Description	Selects the element whose projection is greatest.

```
["a";"bbb";"cc"] |> List.maxBy String.length // "bbb"
```

List.min

Usage	List.min list
Inputs	non-empty comparable list
Output	minimum element
Description	Returns smallest element using generic comparison.

```
List.min [3;1;2] // 1
```

List.minBy

Usage	List.minBy projection list
Inputs	projection; non-empty list
Output	element with minimum projected key
Description	Selects the element whose projection is smallest.

```
["aa";"b"] |> List.minBy String.length // "b"
```

List.ofArray

Usage	List.ofArray array
Inputs	array
Output	list
Description	Converts an array to a list.

```
List.ofArray [|1;2|] // [1;2]
```

List.ofSeq

Usage	List.ofSeq source
Inputs	seq
Output	list
Description	Enumerates a sequence and stores the values in a list.

```
List.ofSeq (seq {1..3}) // [1;2;3]
```

List.pairwise

Usage	List.pairwise list
Inputs	list
Output	list of adjacent pairs
Description	Returns pairs of consecutive elements.

```
[1;2;4] |> List.pairwise // [(1,2);(2,4)]
```

List.partition

Usage	List.partition predicate list
Inputs	predicate; list
Output	matching list * nonmatching list
Description	Splits into elements satisfying and not satisfying predicate.

```
[1..5] |> List.partition (fun x -> x%2=0) // ([2;4],[1;3;5])
```

List.permute

Usage	List.permute indexMap list
Inputs	indexMap: oldIndex -> newIndex; list
Output	permuted list
Description	Reorders elements according to a permutation function.

```
["a";"b"] |> List.permute (fun i -> 1-i) // ["b";"a"]
```

List.pick

Usage	List.pick chooser list
Inputs	chooser: element -> option; list
Output	first chosen value
Description	Applies chooser until Some value is found, then returns it. Raises if none.

```
["x";"42"] |> List.pick (fun s -> match System.Int32.TryParse s with true,v -> Some v | _ -> None) // 42
```

List.randomChoice

Usage	List.randomChoice list
Inputs	non-empty list
Output	one random element
Description	Returns a randomly selected element.

```
["red";"blue"]|> List.randomChoice
```

List.randomChoices

Usage	List.randomChoices count list
Inputs	count; non-empty list
Output	list of random elements
Description	Returns count random selections, typically with replacement.

```
[1;2;3] |> List.randomChoices 5
```

List.randomShuffle

Usage	List.randomShuffle list
Inputs	list
Output	shuffled list
Description	Returns a randomly shuffled copy.

```
[1..5] |> List.randomShuffle
```

List.reduce

Usage	List.reduce reduction list
Inputs	reduction: elt -> elt -> elt; non-empty list
Output	single reduced value
Description	Like fold without an explicit initial state; first element starts the accumulator.

```
[1;2;3] |> List.reduce (+) // 6
```

List.reduceBack

Usage	List.reduceBack reduction list
Inputs	reduction; non-empty list
Output	single value
Description	Right-to-left reduction without explicit state.

```
List.reduceBack (-) [1;2;3] // 2
```

List.removeAt

Usage	List.removeAt index list
Inputs	index; list
Output	new list
Description	Removes one element at index.

```
[1;2;3] |> List.removeAt 1 // [1;3]
```

List.removeManyAt

Usage	List.removeManyAt index count list
Inputs	start index; count; list

Output	new list
Description	Removes count elements starting at index.

```
[1;2;3;4] |> List.removeManyAt 1 2 // [1;4]
```

List.replicate

Usage	List.replicate count value
Inputs	count; value
Output	list
Description	Creates a list with count copies of value.

```
List.replicate 3 "x" // ["x";"x";"x"]
```

List.rev

Usage	List.rev list
Inputs	list
Output	reversed list
Description	Returns elements in reverse order.

```
List.rev [1;2;3] // [3;2;1]
```

List.scan

Usage	List.scan folder state list
Inputs	folder; initial state; list
Output	list of intermediate states
Description	Like fold, but returns every accumulator state including the initial state.

```
[1;2;3] |> List.scan (+) 0 // [0;1;3;6]
```

List.scanBack

Usage	List.scanBack folder list state
Inputs	folder; list; final state
Output	list of intermediate states
Description	Right-to-left scan.

```
List.scanBack (+) [1;2;3] 0 // [6;5;3;0]
```

List.singleton

Usage	List.singleton value
Inputs	value
Output	one-element list
Description	Creates a list containing exactly one value.

```
List.singleton 7 // [7]
```

List.sort

Usage	List.sort list
Inputs	comparable list
Output	sorted list
Description	Sorts ascending by generic comparison.

```
List.sort [3;1;2] // [1;2;3]
```

List.sortBy

Usage	List.sortBy projection list
Inputs	projection; list
Output	sorted list
Description	Sorts ascending by projected key.

```
["bbb";"a";"cc"] |> List.sortBy String.length // ["a";"cc";"bbb"]
```

List.sortByDescending

Usage	List.sortByDescending projection list
Inputs	projection; list
Output	sorted list
Description	Sorts descending by projected key.

```
["a";"bbb";"cc"] |> List.sortByDescending String.length
```

List.sortDescending

Usage	List.sortDescending list
Inputs	comparable list
Output	sorted list
Description	Sorts descending by generic comparison.

```
List.sortDescending [1;3;2] // [3;2;1]
```

List.sortWith

Usage	List.sortWith comparer list
Inputs	comparer: elt -> elt -> int; list
Output	sorted list
Description	Sorts using a custom comparison function.

```
["a";"bbb"] |> List.sortWith (fun a b -> compare b.Length a.Length)
```

List.splitAt

Usage	List.splitAt index list
Inputs	index; list
Output	prefix * suffix
Description	Splits at index.

```
[1;2;3] |> List.splitAt 2 // ([1;2],[3])
```

List.splitInto

Usage	List.splitInto count list
Inputs	number of parts; list
Output	list of lists
Description	Splits into at most count chunks with sizes as even as possible.

```
[1..5] |> List.splitInto 2 // [[1;2;3];[4;5]]
```

List.sum

Usage	List.sum list
Inputs	numeric list
Output	sum
Description	Adds all elements, returning zero for empty lists of supported numeric types.

```
List.sum [1;2;3] // 6
```

List.sumBy

Usage	List.sumBy projection list
Inputs	projection: element -> numeric; list
Output	sum
Description	Projects each element and sums the results.

```
["a";"bb"] |> List.sumBy String.length // 3
```

List.tail

Usage	List.tail list
Inputs	non-empty list
Output	list without first element
Description	Returns all elements after the head. Raises for empty list.

```
List.tail [1;2;3] // [2;3]
```

List.take

Usage	List.take count list
Inputs	count; list
Output	prefix list
Description	Returns first count elements. Raises if too few.

```
[1..5] |> List.take 3 // [1;2;3]
```

List.takeWhile

Usage	List.takeWhile predicate list
Inputs	predicate; list
Output	prefix list
Description	Returns elements from the start while predicate is true.

```
[1;2;3;1] |> List.takeWhile (fun x -> x < 3) // [1;2]
```

List.toArray

Usage	List.toArray list
Inputs	list
Output	array
Description	Converts list to array.

```
[1;2] |> List.toArray
```

List.toSeq

Usage	List.toSeq list
Inputs	list
Output	seq
Description	Views/converts list as a sequence.

```
[1;2] |> List.toSeq
```

List.transpose

Usage	List.transpose lists
Inputs	list of equal-length lists
Output	transposed list of lists
Description	Swaps rows and columns.

```
List.transpose [[1;2];[3;4]] // [[1;3];[2;4]]
```

List.truncate

Usage	List.truncate count list
Inputs	count; list
Output	prefix list
Description	Returns at most count elements; does not fail if too few.

```
[1;2] |> List.truncate 5 // [1;2]
```

List.tryExactlyOne

Usage	List.tryExactlyOne list
Inputs	list
Output	option element
Description	Returns Some x if the list has exactly one element, otherwise None.

```
List.tryExactlyOne [1] // Some 1
```

List.tryFind

Usage	List.tryFind predicate list
Inputs	predicate; list
Output	option element
Description	Optional version of find.

```
[1;2] |> List.tryFind (fun x -> x > 5) // None
```

List.tryFindBack

Usage	List.tryFindBack predicate list
Inputs	predicate; list
Output	option element
Description	Optional version of findBack.

```
[1;2;3] |> List.tryFindBack (fun x -> x < 3) // Some 2
```

List.tryFindIndex

Usage	List.tryFindIndex predicate list
Inputs	predicate; list
Output	option int
Description	Optional version of findIndex.

```
["a"] |> List.tryFindIndex ((=) "b") // None
```

List.tryFindIndexBack

Usage	List.tryFindIndexBack predicate list
Inputs	predicate; list
Output	option int
Description	Optional version of findIndexBack.

```
[1;2;1] |> List.tryFindIndexBack ((=) 1) // Some 2
```

List.tryHead

Usage	List.tryHead list
Inputs	list
Output	option element
Description	Returns Some first element or None for empty list.

```
List.tryHead [] // None
```

List.tryItem

Usage	List.tryItem index list
Inputs	index; list

Output	option element
Description	Returns Some element at index, or None if out of range.

```
["a"] |> List.tryItem 2    // None
```

List.tryLast

Usage	List.tryLast list
Inputs	list
Output	option element
Description	Returns Some last element or None for empty list.

```
List.tryLast [1;2]    // Some 2
```

List.tryPick

Usage	List.tryPick chooser list
Inputs	chooser; list
Output	option chosen value
Description	Optional version of pick.

```
["x"] |> List.tryPick (fun _ -> None)    // None
```

List.unfold

Usage	List.unfold generator state
Inputs	generator: state -> option(value * state); initial state
Output	generated list
Description	Builds a list by repeatedly applying generator until it returns None.

```
List.unfold (fun s -> if s > 3 then None else Some(s, s+1)) 1    // [1;2;3]
```

List.unzip

Usage	List.unzip pairs
Inputs	list of pairs
Output	two lists
Description	Splits a list of pairs into two lists.

```
[(1,"a");(2,"b")] |> List.unzip    // ([1;2],["a";"b"])
```

List.unzip3

Usage	List.unzip3 triples
Inputs	list of triples
Output	three lists
Description	Splits triples into three lists.

```
[(1,"a",true)] |> List.unzip3
```

List.where

Usage	List.where predicate list
Inputs	predicate; list
Output	filtered list
Description	Alias for List.filter.

```
[1..4] |> List.where (fun x -> x>2)    // [3;4]
```

List.windowed

Usage	List.windowed size list
Inputs	window size; list
Output	list of windows
Description	Returns sliding windows of the given size.

```
[1;2;3;4] |> List.windowed 3    // [[1;2;3];[2;3;4]]
```

List.zip

Usage	List.zip list1 list2
Inputs	two equal-length lists
Output	list of pairs
Description	Pairs corresponding elements.

```
List.zip [1;2] ["a";"b"]    // [(1,"a");(2,"b")]
```

List.zip3

Usage	List.zip3 list1 list2 list3
Inputs	three equal-length lists
Output	list of triples
Description	Combines corresponding elements into triples.

```
List.zip3 [1] ["a"] [true] // [(1,"a",true)]
```

Set Module

Set works with immutable collections of unique comparable values. Ordering is by F# generic comparison, not insertion order.

Set.add

Usage	Set.add value set
Inputs	value; input set
Output	new set
Description	Adds a value, returning a new set. Duplicate values have no effect.

```
Set.empty |> Set.add 1 |> Set.add 1 // set [1]
```

Set.contains

Usage	Set.contains element set
Inputs	element; set
Output	bool
Description	Tests membership.

```
Set.contains 2 (set [1;2;3]) // true
```

Set.count

Usage	Set.count set
Inputs	set
Output	int
Description	Returns number of unique elements.

```
Set.count (set [1;2;2]) // 2
```

Set.difference

Usage	Set.difference set1 set2
Inputs	two sets
Output	set
Description	Returns elements in set1 that are not in set2.

```
Set.difference (set [1;2;3]) (set [2]) // set [1;3]
```

Set.empty

Usage	Set.empty
Inputs	no input
Output	empty set
Description	The empty set for any comparable element type.

```
let s : Set<int> = Set.empty
```

Set.exists

Usage	Set.exists predicate set
Inputs	predicate: element -> bool; set
Output	bool
Description	True if any element satisfies predicate. Higher-order function.

```
set [1;2;3] |> Set.exists (fun x -> x > 2) // true
```

Set.filter

Usage	Set.filter predicate set
Inputs	predicate; set
Output	set
Description	Keeps elements satisfying predicate. Higher-order function.

```
set [1..5] |> Set.filter (fun x -> x%2=0) // set [2;4]
```

Set.fold

Usage	Set.fold folder state set
Inputs	folder: state -> element -> state; state; set
Output	final state
Description	Folds through set elements in sorted order. Higher-order function.

```
set [1;2;3] |> Set.fold (+) 0 // 6
```

Set.foldBack

Usage	Set.foldBack folder set state
Inputs	folder: element -> state -> state; set; state
Output	final state

Description	Right-to-left fold over set order.
-------------	------------------------------------

```
Set.foldBack (fun x acc -> x::acc) (set [1;2]) []
```

Set.forall

Usage	Set.forall predicate set
Inputs	predicate; set
Output	bool
Description	True if all elements satisfy predicate.

```
set [2;4] |> Set.forall (fun x -> x%2=0) // true
```

Set.intersect

Usage	Set.intersect set1 set2
Inputs	two sets
Output	set
Description	Returns values present in both sets.

```
Set.intersect (set [1;2]) (set [2;3]) // set [2]
```

Set.intersectMany

Usage	Set.intersectMany sets
Inputs	sequence of sets
Output	set
Description	Returns intersection across all supplied sets.

```
Set.intersectMany [set [1;2]; set [2;3]; set [2]] // set [2]
```

Set.isEmpty

Usage	Set.isEmpty set
Inputs	set
Output	bool
Description	True if the set contains no elements.

```
Set.isEmpty Set.empty // true
```

Set.isProperSubset

Usage	Set.isProperSubset set1 set2
Inputs	two sets
Output	bool
Description	True if set1 is a strict subset of set2.

```
Set.isProperSubset (set [1]) (set [1;2]) // true
```

Set.isProperSuperset

Usage	Set.isProperSuperset set1 set2
Inputs	two sets
Output	bool
Description	True if set1 strictly contains set2.

```
Set.isProperSuperset (set [1;2]) (set [1]) // true
```

Set.isSubset

Usage	Set.isSubset set1 set2
Inputs	two sets
Output	bool
Description	True if every element of set1 is in set2.

```
Set.isSubset (set [1]) (set [1;2]) // true
```

Set.isSuperset

Usage	Set.isSuperset set1 set2
Inputs	two sets
Output	bool
Description	True if set1 contains all elements of set2.

```
Set.isSuperset (set [1;2]) (set [1]) // true
```

Set.iterate

Usage	Set.iter action set
Inputs	action: element -> unit; set
Output	unit
Description	Applies an action to each element in set order. Higher-order function.

```
set [1;2] |> Set.iter (printfn "%d")
```

Set.map

Usage	Set.map mapping set
Inputs	mapping: element -> result; set
Output	set
Description	Transforms each element and removes duplicates in the mapped results.

```
set [1;2;3] |> Set.map (fun x -> x%2) // set [0;1]
```

Set.maxElement

Usage	Set.maxElement set
Inputs	non-empty set
Output	maximum element
Description	Returns greatest element by comparison.

```
Set.maxElement (set [3;1;2]) // 3
```

Set.minElement

Usage	Set.minElement set
Inputs	non-empty set
Output	minimum element
Description	Returns smallest element by comparison.

```
Set.minElement (set [3;1;2]) // 1
```

Set.ofArray

Usage	Set.ofArray array
Inputs	array
Output	set
Description	Builds a set from array values.

```
Set.ofArray [|1;2;1|] // set [1;2]
```

Set.ofList

Usage	Set.ofList list
Inputs	list
Output	set
Description	Builds a set from list values.

```
Set.ofList [1;2;1] // set [1;2]
```

Set.ofSeq

Usage	Set.ofSeq source
Inputs	sequence
Output	set
Description	Builds a set from a sequence.

```
Set.ofSeq (seq {1;2;1}) // set [1;2]
```

Set.partition

Usage	Set.partition predicate set
Inputs	predicate; set
Output	matching set * nonmatching set
Description	Splits a set into elements that pass and fail predicate.

```
set [1..4] |> Set.partition (fun x -> x%2=0)
```

Set.remove

Usage	Set.remove value set
Inputs	value; set
Output	new set
Description	Returns a new set without value. No error if absent.

```
set [1;2] |> Set.remove 1 // set [2]
```

Set.singleton

Usage	Set.singleton value
Inputs	value
Output	one-element set
Description	Creates a set containing one value.

```
Set.singleton "x" // set ["x"]
```

Set.toArray

Usage	Set.toArray set
Inputs	set
Output	array
Description	Returns set elements as an array in sorted order.

```
Set.toArray (set [2;1]) // [|1;2|]
```

Set.toList

Usage	Set.toList set
Inputs	set
Output	list
Description	Returns set elements as a list in sorted order.

```
Set.toList (set [2;1]) // [1;2]
```

Set.toSeq

Usage	Set.toSeq set
Inputs	set
Output	sequence
Description	Views set elements as a sequence in sorted order.

```
Set.toSeq (set [1;2])
```

Set.union

Usage	Set.union set1 set2
Inputs	two sets
Output	set
Description	Returns elements from either set.

```
Set.union (set [1;2]) (set [2;3]) // set [1;2;3]
```

Set.unionMany

Usage	Set.unionMany sets
Inputs	sequence of sets
Output	set
Description	Returns the union of all supplied sets.

```
Set.unionMany [set [1]; set [2;3]] // set [1;2;3]
```

Map Module

Map works with immutable key/value dictionaries where keys require comparison. Operations return new maps when adding/removing/changing.

Map.add

Usage	Map.add key value table
Inputs	key; value; map
Output	new map
Description	Adds or replaces a key/value binding.

```
Map.empty |> Map.add "a" 1 |> Map.add "a" 2 // map [("a",2)]
```

Map.change

Usage	Map.change key f table
Inputs	key; f: option value -> option value; map
Output	new map
Description	Adds, updates, or removes a binding depending on the option returned by f.

```
Map.empty |> Map.change "a" (function None -> Some 1 | Some v -> Some(v+1))
```

Map.containsKey

Usage	Map.containsKey key table
Inputs	key; map
Output	bool
Description	Returns true if the key is bound.

```
Map.containsKey "a" (Map ["a",1]) // true
```

Map.count

Usage	Map.count table
Inputs	map
Output	int
Description	Number of key/value bindings.

```
Map.count (Map ["a",1; "b",2]) // 2
```

Map.empty

Usage	Map.empty
Inputs	no input
Output	empty map
Description	The empty map for comparable keys.

```
let m : Map<string,int> = Map.empty
```

Map.exists

Usage	Map.exists predicate table
Inputs	predicate: key -> value -> bool; map
Output	bool
Description	True if any binding satisfies predicate. Higher-order function.

```
Map ["a",1;"b",2] |> Map.exists (fun k v -> v>1) // true
```

Map.filter

Usage	Map.filter predicate table
Inputs	predicate: key -> value -> bool; map
Output	map
Description	Keeps bindings satisfying predicate. Higher-order function.

```
Map ["a",1;"b",2] |> Map.filter (fun _ v -> v>1)
```

Map.find

Usage	Map.find key table
Inputs	key; map
Output	value
Description	Returns value for key. Raises if absent.

```
Map.find "a" (Map ["a",1]) // 1
```

Map.findKey

Usage	Map.findKey predicate table
Inputs	predicate: key -> value -> bool; map

Output	key
Description	Returns first key whose binding satisfies predicate. Raises if none.

```
Map ["a",1;"b",2] |> Map.findKey (fun _ v -> v=2) // "b"
```

Map.fold

Usage	Map.fold folder state table
Inputs	folder: state -> key -> value -> state; state; map
Output	final state
Description	Folds over bindings in key order. Higher-order function.

```
Map ["a",1;"b",2] |> Map.fold (fun acc _ v -> acc+v) 0 // 3
```

Map.forall

Usage	Map.forall predicate table
Inputs	predicate: key -> value -> bool; map
Output	bool
Description	True if every binding satisfies predicate.

```
Map ["a",1] |> Map.forall (fun _ v -> v>0) // true
```

Map.isEmpty

Usage	Map.isEmpty table
Inputs	map
Output	bool
Description	True if the map has no bindings.

```
Map.isEmpty Map.empty // true
```

Map.iterate

Usage	Map.iter action table
Inputs	action: key -> value -> unit; map
Output	unit
Description	Applies an action to each binding in key order.

```
Map ["a",1] |> Map.iter (fun k v -> printfn "%s=%d" k v)
```

Map.keys

Usage	Map.keys table
Inputs	map
Output	sequence of keys
Description	Returns all keys in key order.

```
Map ["a",1;"b",2] |> Map.keys
```

Map.map

Usage	Map.map mapping table
Inputs	mapping: key -> value -> result; map
Output	map with same keys and new values
Description	Transforms values while preserving keys. Higher-order function.

```
Map ["a",1] |> Map.map (fun k v -> k + string v) // map [("a","a1")]
```

Map.ofArray

Usage	Map.ofArray array
Inputs	array of key/value tuples
Output	map
Description	Builds a map; duplicate keys keep the later binding according to construction semantics.

```
Map.ofArray [| "a",1; "b",2 |]
```

Map.ofList

Usage	Map.ofList list
Inputs	list of key/value tuples
Output	map
Description	Builds a map from a list of pairs.

```
Map.ofList ["a",1; "b",2]
```

Map.ofSeq

Usage	Map.ofSeq source
Inputs	sequence of key/value tuples
Output	map
Description	Builds a map from a sequence of pairs.

```
Map.ofSeq (seq { "a",1; "b",2 })
```

Map.partition

Usage	Map.partition predicate table
Inputs	predicate: key -> value -> bool; map
Output	matching map * nonmatching map
Description	Splits bindings by predicate.

```
Map ["a",1;"b",2] |> Map.partition (fun _ v -> v%2=0)
```

Map.pick

Usage	Map.pick chooser table
Inputs	chooser: key -> value -> option; map
Output	chosen value
Description	Returns first Some produced by chooser. Raises if none.

```
Map ["a",1;"b",2] |> Map.pick (fun k v -> if v=2 then Some k else None)
```

Map.remove

Usage	Map.remove key table
Inputs	key; map
Output	new map
Description	Returns map without the key. No error if absent.

```
Map ["a",1;"b",2] |> Map.remove "a"
```

Map.toArray

Usage	Map.toArray table
Inputs	map
Output	array of key/value pairs
Description	Converts bindings to an array in key order.

```
Map ["a",1] |> Map.toArray
```

Map.toList

Usage	Map.toList table
Inputs	map
Output	list of key/value pairs
Description	Converts bindings to a list in key order.

```
Map ["a",1] |> Map.toList
```

Map.toSeq

Usage	Map.toSeq table
Inputs	map
Output	sequence of key/value pairs
Description	Views bindings as a sequence in key order.

```
Map ["a",1] |> Map.toSeq
```

Map.tryFind

Usage	Map.tryFind key table
Inputs	key; map
Output	option value
Description	Optional version of find.

```
Map.tryFind "z" (Map ["a",1]) // None
```

Map.tryFindKey

Usage	Map.tryFindKey predicate table
Inputs	predicate: key -> value -> bool; map
Output	option key
Description	Optional version of findKey.

```
Map ["a",1] |> Map.tryFindKey (fun _ v -> v=9) // None
```

Map.tryPick

Usage	Map.tryPick chooser table
Inputs	chooser; map
Output	option chosen value
Description	Optional version of pick.

```
Map ["a",1] |> Map.tryPick (fun _ _ -> None) // None
```

Map.values

Usage	Map.values table
Inputs	map
Output	sequence of values
Description	Returns all values in key order.

Map ["a", 1; "b", 2] |> Map.values

Seq Module

Seq works with IEnumerable-style sequences. Many functions are lazy and can represent streaming or infinite data, but repeated enumeration may recompute unless cached.

Seq.allPairs

Usage	Seq.allPairs source1 source2
Inputs	sequence/source1: first sequence/source; sequence/source2: second sequence/source
Output	sequence of tuples containing every pairing
Description	Creates the Cartesian product of two sequences, preserving nested-loop order: each element of sequence1 is paired with each element of sequence2.

```
Seq.allPairs [1;2] ["a";"b"] // [(1,"a"); (1,"b"); (2,"a"); (2,"b")]
```

Seq.append

Usage	Seq.append source1 source2
Inputs	two sequence/sources of the same element type
Output	one sequence
Description	Returns a new sequence containing sequence1 followed by sequence2. The original sequences are unchanged.

```
Seq.append [1;2] [3;4] // [1;2;3;4]
```

Seq.average

Usage	Seq.average source
Inputs	numeric sequence/source
Output	average value
Description	Computes the arithmetic mean. Fails for an empty sequence.

```
Seq.average [1.0; 2.0; 3.0] // 2.0
```

Seq.averageBy

Usage	Seq.averageBy projection source
Inputs	projection: element -> numeric value; sequence/source
Output	average of projected values
Description	Maps each element to a numeric value and averages those values. Higher-order function.

```
["aa"; "bbbb"] |> Seq.averageBy (fun s -> float s.Length) // 3.0
```

Seq.cache

Usage	Seq.cache source
Inputs	source sequence
Output	cached sequence
Description	Returns a sequence that caches elements as they are enumerated, avoiding repeated recomputation on later enumerations.

```
let cached = Seq.init 3 (fun i -> printfn "gen"; i) |> Seq.cache
```

Seq.cast

Usage	Seq.cast source
Inputs	untyped IEnumerable
Output	typed sequence
Description	Casts each element of a non-generic enumerable to the target type.

```
let xs = (System.Collections.ArrayList([1;2])) |> Seq.cast<int>
```

Seq.choose

Usage	Seq.choose chooser source
Inputs	chooser: element -> option; sequence/source
Output	sequence of unwrapped Some results
Description	Combines filtering and mapping: the chooser returns Some value to keep or None to drop. Higher-order function.

```
["1"; "x"; "2"] |> Seq.choose (fun s -> match System.Int32.TryParse s with true,v -> Some v | _ -> None) // [1;2]
```

Seq.chunkBySize

Usage	Seq.chunkBySize size source
Inputs	size: positive chunk size; sequence/source
Output	sequence of sequences
Description	Splits the sequence into chunks of at most size elements.

```
[1..7] |> Seq.chunkBySize 3 // [[1;2;3]; [4;5;6]; [7]]
```

Seq.collect

Usage	Seq.collect mapping source
Inputs	mapping: element -> sequence/source; sequence/source

Output	concatenated sequence
Description	Maps each element to a sequence and concatenates the results. Also called flatMap/bind. Higher-order function.

```
[1;2;3] |> Seq.collect (fun x -> [x; x*x])    // [1;1;2;4;3;9]
```

Seq.compareWith

Usage	Seq.compareWith comparer source1 source2
Inputs	comparer: elt -> elt -> int; two sequence/sources
Output	negative, zero, or positive int
Description	Lexicographically compares two sequences using the supplied element comparison. Higher-order function.

```
Seq.compareWith compare [1;2] [1;3]    // negative
```

Seq.concat

Usage	Seq.concat sources
Inputs	sequence/sequence/source of sequence/sources
Output	single flattened sequence
Description	Concatenates all inner sequences in order.

```
Seq.concat [[1;2]; [3]; []]    // [1;2;3]
```

Seq.contains

Usage	Seq.contains value source
Inputs	value to test; sequence/source
Output	bool
Description	Returns true when any element equals the value.

```
[1;2;3] |> Seq.contains 2    // true
```

Seq.countBy

Usage	Seq.countBy projection source
Inputs	projection: element -> key; sequence/source
Output	sequence of key * count pairs
Description	Groups by a projected key and counts elements in each group. Higher-order function.

```
["a";"bb";"c"] |> Seq.countBy String.length    // [(1,2); (2,1)]
```

Seq.delay

Usage	Seq.delay generator
Inputs	generator: unit -> seq
Output	deferred sequence
Description	Creates a sequence whose underlying source is produced when enumerated.

```
let s = Seq.delay (fun () -> seq { 1; 2 })
```

Seq.distinct

Usage	Seq.distinct source
Inputs	source sequence
Output	sequence without duplicates
Description	Lazily yields first occurrences only.

```
seq [1;2;1] |> Seq.distinct
```

Seq.distinctBy

Usage	Seq.distinctBy projection source
Inputs	projection: element -> key; sequence/source
Output	sequence
Description	Removes duplicates based on projected keys, keeping first occurrence per key. Higher-order function.

```
["AL";"bob";"Amy"] |> Seq.distinctBy (fun s -> s.[0])    // ["AL"; "bob"]
```

Seq.empty

Usage	Seq.empty
Inputs	no input
Output	empty sequence
Description	The empty sequence.

```
let s : seq<int> = Seq.empty
```

Seq.exactlyOne

Usage	Seq.exactlyOne source
Inputs	sequence/source
Output	single element
Description	Returns the only element. Raises if the sequence is empty or has more than one element.

```
Seq.exactlyOne [42] // 42
```

Seq.except

Usage	Seq.except exclusions source
Inputs	exclusions: seq of values; sequence/source
Output	filtered sequence
Description	Returns elements whose values are not in exclusions.

```
[1;2;3;4] |> Seq.except [2;4] // [1;3]
```

Seq.exists

Usage	Seq.exists predicate source
Inputs	predicate: element -> bool; sequence/source
Output	bool
Description	Returns true if at least one element satisfies the predicate. Short-circuits. Higher-order function.

```
[1;2;3] |> Seq.exists (fun x -> x > 2) // true
```

Seq.exists2

Usage	Seq.exists2 predicate source1 source2
Inputs	predicate: elt1 -> elt2 -> bool; two equal-length sequence/sources
Output	bool
Description	Tests paired elements and returns true if any pair satisfies the predicate. Higher-order function.

```
Seq.exists2 (fun a b -> a+b=10) [1;4] [9;2] // true
```

Seq.filter

Usage	Seq.filter predicate source
Inputs	predicate: element -> bool; sequence/source
Output	filtered sequence
Description	Keeps only elements for which the predicate returns true. Higher-order function.

```
[1..6] |> Seq.filter (fun x -> x % 2 = 0) // [2;4;6]
```

Seq.find

Usage	Seq.find predicate source
Inputs	predicate; sequence/source
Output	first matching element
Description	Returns the first element satisfying predicate. Raises if none match.

```
[1..5] |> Seq.find (fun x -> x > 3) // 4
```

Seq.findBack

Usage	Seq.findBack predicate source
Inputs	predicate; sequence/source
Output	last matching element
Description	Searches from the end and returns the last element satisfying predicate. Raises if none.

```
[1..5] |> Seq.findBack (fun x -> x < 5) // 4
```

Seq.findIndex

Usage	Seq.findIndex predicate source
Inputs	predicate; sequence/source
Output	zero-based index
Description	Returns the index of the first matching element. Raises if none.

```
["a";"bb"] |> Seq.findIndex (fun s -> s.Length=2) // 1
```

Seq.findIndexBack

Usage	Seq.findIndexBack predicate source
Inputs	predicate; sequence/source
Output	zero-based index
Description	Returns the index of the last matching element. Raises if none.

```
[1;2;3;2] |> Seq.findIndexBack ((=) 2) // 3
```

Seq.fold

Usage	Seq.fold folder state source
Inputs	folder: state -> element -> state; initial state; sequence/source
Output	final state
Description	Traverses left to right, threading an accumulator. Higher-order function.

```
[1;2;3] |> Seq.fold (fun acc x -> acc + x) 0 // 6
```

Seq.fold2

Usage	Seq.fold2 folder state source1 source2
Inputs	folder: state -> elt1 -> elt2 -> state; state; two equal-length sequence/sources
Output	final state
Description	Folds over paired elements from two sequences. Higher-order function.

```
Seq.fold2 (fun acc a b -> acc + a*b) 0 [1;2] [10;20] // 50
```

Seq.foldBack

Usage	Seq.foldBack folder source state
Inputs	folder: element -> state -> state; sequence/source; state
Output	final state
Description	Traverses right to left, threading an accumulator. Higher-order function.

```
Seq.foldBack (fun x acc -> x::acc) [1;2] [] // [1;2]
```

Seq.foldBack2

Usage	Seq.foldBack2 folder source1 source2 state
Inputs	folder: elt1 -> elt2 -> state -> state; sequence/sources; state
Output	final state
Description	Right-to-left fold over paired elements from two sequences. Higher-order function.

```
Seq.foldBack2 (fun a b acc -> (a,b)::acc) [1;2] [3;4] [] // [(1,3);(2,4)]
```

Seq.forall

Usage	Seq.forall predicate source
Inputs	predicate; sequence/source
Output	bool
Description	Returns true if every element satisfies predicate. Short-circuits.

```
[2;4;6] |> Seq.forall (fun x -> x % 2 = 0) // true
```

Seq.forall2

Usage	Seq.forall2 predicate source1 source2
Inputs	predicate over pairs; two equal-length sequence/sources
Output	bool
Description	Returns true if every pair satisfies predicate.

```
Seq.forall2 (=) [1;2] [1;2] // true
```

Seq.groupBy

Usage	Seq.groupBy projection source
Inputs	projection: element -> key; sequence/source
Output	sequence of key * group sequence
Description	Groups elements by a projected key. Higher-order function.

```
[ "a"; "bb"; "c" ] |> Seq.groupBy String.length // [(1,["a";"c"]); (2,["bb"])]
```

Seq.head

Usage	Seq.head source
Inputs	non-empty sequence/source
Output	first element
Description	Returns first element. Raises for empty sequence.

```
Seq.head [10;20] // 10
```

Seq.indexed

Usage	Seq.indexed source
Inputs	sequence/source
Output	sequence of index * element
Description	Pairs each element with its zero-based index.

```
Seq.indexed [ "a"; "b" ] // [(0, "a"); (1, "b")]
```

Seq.init

Usage	Seq.init count initializer
Inputs	count; initializer: index -> element
Output	new sequence
Description	Creates a sequence by calling initializer for indexes 0..count-1. Higher-order function.

```
Seq.init 4 (fun i -> i*i) // [0;1;4;9]
```

Seq.initInfinite

Usage	Seq.initInfinite initializer
Inputs	initializer: index -> value

Output	infinite sequence
Description	Creates an infinite sequence by applying initializer to indexes.

```
Seq.initInfinite (fun i -> i*i) |> Seq.take 5
```

Seq.isEmpty

Usage	Seq.isEmpty source
Inputs	source sequence
Output	bool
Description	Returns true if the sequence has no elements. May inspect at most the first element.

```
Seq.isEmpty Seq.empty // true
```

Seq.item

Usage	Seq.item index source
Inputs	zero-based index; sequence/source
Output	element
Description	Returns the element at index. Raises if out of range.

```
["a";"b"] |> Seq.item 1 // "b"
```

Seq.iter

Usage	Seq.iter action source
Inputs	action: element -> unit; sequence/source
Output	unit
Description	Applies an action to each element for side effects. Higher-order function.

```
[1;2] |> Seq.iter (printfn "%d")
```

Seq.iter2

Usage	Seq.iter2 action source1 source2
Inputs	action over pairs; equal-length sequence/sources
Output	unit
Description	Applies an action to paired elements.

```
Seq.iter2 (fun a b -> printfn "%d:%s" a b) [1;2] ["a";"b"]
```

Seq.iteri

Usage	Seq.iteri action source
Inputs	action: index -> element -> unit; sequence/source
Output	unit
Description	Applies an indexed action to each element.

```
["a";"b"] |> Seq.iteri (fun i x -> printfn "%d=%s" i x)
```

Seq.iteri2

Usage	Seq.iteri2 action source1 source2
Inputs	action: index -> elt1 -> elt2 -> unit; sequence/sources
Output	unit
Description	Applies an indexed action to paired elements.

```
Seq.iteri2 (fun i a b -> printfn "%d %A %A" i a b) [1] [2]
```

Seq.last

Usage	Seq.last source
Inputs	non-empty sequence/source
Output	last element
Description	Returns last element. Raises for empty sequence.

```
Seq.last [1;2;3] // 3
```

Seq.length

Usage	Seq.length source
Inputs	sequence/source
Output	int
Description	Returns number of elements.

```
Seq.length [1;2;3] // 3
```

Seq.map

Usage	Seq.map mapping source
Inputs	mapping: element -> result; sequence/source
Output	mapped sequence
Description	Transforms each element. Higher-order function.

```
[1;2;3] |> Seq.map (fun x -> x*x)    // [1;4;9]
```

Seq.map2

Usage	Seq.map2 mapping source1 source2
Inputs	mapping: elt1 -> elt2 -> result; equal-length sequence/sources
Output	mapped sequence
Description	Transforms paired elements from two sequences.

```
Seq.map2 (+) [1;2] [10;20]    // [11;22]
```

Seq.map3

Usage	Seq.map3 mapping source1 source2 source3
Inputs	mapping over triples; equal-length sequence/sources
Output	mapped sequence
Description	Transforms corresponding triples from three sequences.

```
Seq.map3 (fun a b c -> a+b+c) [1] [2] [3]    // [6]
```

Seq.mapi

Usage	Seq.mapi mapping source
Inputs	mapping: index -> element -> result; sequence/source
Output	mapped sequence
Description	Indexed map. Higher-order function.

```
["a";"b"] |> Seq.mapi (fun i x -> string i + x)    // ["0a";"1b"]
```

Seq.mapi2

Usage	Seq.mapi2 mapping source1 source2
Inputs	mapping: index -> elt1 -> elt2 -> result; sequence/sources
Output	mapped sequence
Description	Indexed map over pairs.

```
Seq.mapi2 (fun i a b -> i+a+b) [1] [2]    // [3]
```

Seq.max

Usage	Seq.max source
Inputs	non-empty comparable sequence/source
Output	maximum element
Description	Returns greatest element using generic comparison.

```
Seq.max [3;1;2]    // 3
```

Seq.maxBy

Usage	Seq.maxBy projection source
Inputs	projection; non-empty sequence/source
Output	element with maximum projected key
Description	Selects the element whose projection is greatest.

```
["a";"bbb";"cc"] |> Seq.maxBy String.length    // "bbb"
```

Seq.min

Usage	Seq.min source
Inputs	non-empty comparable sequence/source
Output	minimum element
Description	Returns smallest element using generic comparison.

```
Seq.min [3;1;2]    // 1
```

Seq.minBy

Usage	Seq.minBy projection source
Inputs	projection; non-empty sequence/source
Output	element with minimum projected key
Description	Selects the element whose projection is smallest.

```
["aa";"b"] |> Seq.minBy String.length    // "b"
```

Seq.ofArray

Usage	Seq.ofArray array
Inputs	array
Output	sequence
Description	Creates a sequence over array elements.

```
Seq.ofArray [|1;2|]
```

Seq.ofList

Usage	Seq.ofList list
Inputs	list
Output	sequence
Description	Creates a sequence over list elements.

```
Seq.ofList [1;2]
```

Seq.pairwise

Usage	Seq.pairwise source
Inputs	sequence/source
Output	sequence of adjacent pairs
Description	Returns pairs of consecutive elements.

```
[1;2;4] |> Seq.pairwise // [(1,2);(2,4)]
```

Seq.partition

Usage	Seq.partition predicate source
Inputs	predicate; sequence/source
Output	matching sequence * nonmatching sequence
Description	Splits into elements satisfying and not satisfying predicate.

```
[1..5] |> Seq.partition (fun x -> x%2=0) // ([2;4],[1;3;5])
```

Seq.permute

Usage	Seq.permute indexMap source
Inputs	indexMap: oldIndex -> newIndex; sequence/source
Output	permuted sequence
Description	Reorders elements according to a permutation function.

```
["a";"b"] |> Seq.permute (fun i -> 1-i) // ["b";"a"]
```

Seq.pick

Usage	Seq.pick chooser source
Inputs	chooser: element -> option; sequence/source
Output	first chosen value
Description	Applies chooser until Some value is found, then returns it. Raises if none.

```
["x";"42"] |> Seq.pick (fun s -> match System.Int32.TryParse s with true,v -> Some v | _ -> None) // 42
```

Seq.readonly

Usage	Seq.readonly source
Inputs	source sequence
Output	read-only sequence
Description	Hides a sequence behind a read-only enumerable wrapper.

```
let s = [1;2] |> Seq.readonly
```

Seq.reduce

Usage	Seq.reduce reduction source
Inputs	reduction: elt -> elt -> elt; non-empty sequence/source
Output	single reduced value
Description	Like fold without an explicit initial state; first element starts the accumulator.

```
[1;2;3] |> Seq.reduce (+) // 6
```

Seq.reduceBack

Usage	Seq.reduceBack reduction source
Inputs	reduction; non-empty sequence/source
Output	single value
Description	Right-to-left reduction without explicit state.

```
Seq.reduceBack (-) [1;2;3] // 2
```

Seq.replicate

Usage	Seq.replicate count value
Inputs	count; value
Output	sequence
Description	Creates a sequence with count copies of value.

```
Seq.replicate 3 "x" // ["x";"x";"x"]
```

Seq.rev

Usage	Seq.rev source
Inputs	sequence/source

Output	reversed sequence
Description	Returns elements in reverse order.

```
Seq.rev [1;2;3] // [3;2;1]
```

Seq.scan

Usage	Seq.scan folder state source
Inputs	folder; initial state; sequence/source
Output	sequence of intermediate states
Description	Like fold, but returns every accumulator state including the initial state.

```
[1;2;3] |> Seq.scan (+) 0 // [0;1;3;6]
```

Seq.scanBack

Usage	Seq.scanBack folder source state
Inputs	folder; sequence/source; final state
Output	sequence of intermediate states
Description	Right-to-left scan.

```
Seq.scanBack (+) [1;2;3] 0 // [6;5;3;0]
```

Seq.singleton

Usage	Seq.singleton value
Inputs	value
Output	one-element sequence
Description	Creates a sequence with one element.

```
Seq.singleton 42
```

Seq.sort

Usage	Seq.sort source
Inputs	comparable sequence/source
Output	sorted sequence
Description	Sorts ascending by generic comparison.

```
Seq.sort [3;1;2] // [1;2;3]
```

Seq.sortBy

Usage	Seq.sortBy projection source
Inputs	projection; sequence/source
Output	sorted sequence
Description	Sorts ascending by projected key.

```
["bbb";"a";"cc"] |> Seq.sortBy String.length // ["a";"cc";"bbb"]
```

Seq.sortByDescending

Usage	Seq.sortByDescending projection source
Inputs	projection; sequence/source
Output	sorted sequence
Description	Sorts descending by projected key.

```
["a";"bbb";"cc"] |> Seq.sortByDescending String.length
```

Seq.sortDescending

Usage	Seq.sortDescending source
Inputs	comparable sequence/source
Output	sorted sequence
Description	Sorts descending by generic comparison.

```
Seq.sortDescending [1;3;2] // [3;2;1]
```

Seq.sortWith

Usage	Seq.sortWith comparer source
Inputs	comparer: elt -> elt -> int; sequence/source
Output	sorted sequence
Description	Sorts using a custom comparison function.

```
["a";"bbb"] |> Seq.sortWith (fun a b -> compare b.Length a.Length)
```

Seq.splitAt

Usage	Seq.splitAt index source
Inputs	index; sequence/source
Output	prefix * suffix
Description	Splits at index.

```
[1;2;3] |> Seq.splitAt 2 // ([1;2],[3])
```

Seq.splitInto

Usage	Seq.splitInto count source
Inputs	number of parts; sequence/source
Output	sequence of sequences
Description	Splits into at most count chunks with sizes as even as possible.

```
[1..5] |> Seq.splitInto 2 // [[1;2;3],[4;5]]
```

Seq.sum

Usage	Seq.sum source
Inputs	numeric sequence/source
Output	sum
Description	Adds all elements, returning zero for empty sequences of supported numeric types.

```
Seq.sum [1;2;3] // 6
```

Seq.sumBy

Usage	Seq.sumBy projection source
Inputs	projection: element -> numeric; sequence/source
Output	sum
Description	Projects each element and sums the results.

```
["a","bb"] |> Seq.sumBy String.length // 3
```

Seq.tail

Usage	Seq.tail source
Inputs	non-empty sequence/source
Output	sequence without first element
Description	Returns all elements after the head. Raises for empty sequence.

```
Seq.tail [1;2;3] // [2;3]
```

Seq.take

Usage	Seq.take count source
Inputs	count; sequence/source
Output	prefix sequence
Description	Returns first count elements. Raises if too few.

```
[1..5] |> Seq.take 3 // [1;2;3]
```

Seq.takeExactly

Usage	Seq.takeExactly count source
Inputs	count; source
Output	sequence of exactly count elements
Description	Returns exactly count elements and fails if the source has too few.

```
seq {1..5} |> Seq.takeExactly 3
```

Seq.takeWhile

Usage	Seq.takeWhile predicate source
Inputs	predicate; sequence/source
Output	prefix sequence
Description	Returns elements from the start while predicate is true.

```
[1;2;3;1] |> Seq.takeWhile (fun x -> x < 3) // [1;2]
```

Seq.toArray

Usage	Seq.toArray source
Inputs	source sequence
Output	array
Description	Enumerates the source into an array.

```
seq {1..3} |> Seq.toArray
```

Seq.toList

Usage	Seq.toList source
Inputs	source sequence
Output	list
Description	Enumerates the source into a list.

```
seq {1..3} |> Seq.toList
```

Seq.transpose

Usage	Seq.transpose sources
Inputs	sequence/source of equal-length sequence/sources
Output	transposed sequence of sequences
Description	Swaps rows and columns.

```
Seq.transpose [[1;2];[3;4]] // [[1;3];[2;4]]
```

Seq.truncate

Usage	Seq.truncate count source
Inputs	count; sequence/source
Output	prefix sequence
Description	Returns at most count elements; does not fail if too few.

```
[1;2] |> Seq.truncate 5 // [1;2]
```

Seq.tryExactlyOne

Usage	Seq.tryExactlyOne source
Inputs	sequence/source
Output	option element
Description	Returns Some x if the sequence has exactly one element, otherwise None.

```
Seq.tryExactlyOne [1] // Some 1
```

Seq.tryFind

Usage	Seq.tryFind predicate source
Inputs	predicate; sequence/source
Output	option element
Description	Optional version of find.

```
[1;2] |> Seq.tryFind (fun x -> x > 5) // None
```

Seq.tryFindBack

Usage	Seq.tryFindBack predicate source
Inputs	predicate; sequence/source
Output	option element
Description	Optional version of findBack.

```
[1;2;3] |> Seq.tryFindBack (fun x -> x < 3) // Some 2
```

Seq.tryFindIndex

Usage	Seq.tryFindIndex predicate source
Inputs	predicate; sequence/source
Output	option int
Description	Optional version of findIndex.

```
["a"] |> Seq.tryFindIndex ((=) "b") // None
```

Seq.tryFindIndexBack

Usage	Seq.tryFindIndexBack predicate source
Inputs	predicate; sequence/source
Output	option int
Description	Optional version of findIndexBack.

```
[1;2;1] |> Seq.tryFindIndexBack ((=) 1) // Some 2
```

Seq.tryHead

Usage	Seq.tryHead source
Inputs	sequence/source
Output	option element
Description	Returns Some first element or None for empty sequence.

```
Seq.tryHead [] // None
```

Seq.tryItem

Usage	Seq.tryItem index source
Inputs	index; sequence/source
Output	option element
Description	Returns Some element at index, or None if out of range.

```
["a"] |> Seq.tryItem 2 // None
```

Seq.tryLast

Usage	Seq.tryLast source
Inputs	sequence/source

Output	option element
Description	Returns Some last element or None for empty sequence.

```
Seq.tryLast [1;2] // Some 2
```

Seq.tryPick

Usage	Seq.tryPick chooser source
Inputs	chooser; sequence/source
Output	option chosen value
Description	Optional version of pick.

```
[ "x" ] |> Seq.tryPick (fun _ -> None) // None
```

Seq.unfold

Usage	Seq.unfold generator state
Inputs	generator: state -> option(value * state); initial state
Output	generated sequence
Description	Builds a sequence by repeatedly applying generator until it returns None.

```
Seq.unfold (fun s -> if s > 3 then None else Some(s, s+1)) 1 // [1;2;3]
```

Seq.unzip

Usage	Seq.unzip pairs
Inputs	sequence/source of pairs
Output	two sequences
Description	Splits a sequence of pairs into two sequences.

```
[(1, "a"); (2, "b")] |> Seq.unzip // ([1;2], ["a"; "b"])
```

Seq.unzip3

Usage	Seq.unzip3 triples
Inputs	sequence/source of triples
Output	three sequences
Description	Splits triples into three sequences.

```
[(1, "a", true)] |> Seq.unzip3
```

Seq.where

Usage	Seq.where predicate source
Inputs	predicate; sequence/source
Output	filtered sequence
Description	Alias for List.filter.

```
[1..4] |> Seq.where (fun x -> x>2) // [3;4]
```

Seq.windowed

Usage	Seq.windowed size source
Inputs	window size; sequence/source
Output	sequence of windows
Description	Returns sliding windows of the given size.

```
[1;2;3;4] |> Seq.windowed 3 // [[1;2;3]; [2;3;4]]
```

Seq.zip

Usage	Seq.zip source1 source2
Inputs	two equal-length sequence/sources
Output	sequence of pairs
Description	Pairs corresponding elements.

```
Seq.zip [1;2] ["a"; "b"] // [(1, "a"); (2, "b")]
```

Seq.zip3

Usage	Seq.zip3 source1 source2 source3
Inputs	three equal-length sequence/sources
Output	sequence of triples
Description	Combines corresponding elements into triples.

```
Seq.zip3 [1] ["a"] [true] // [(1, "a", true)]
```


Source URLs

FSharp.Core List Module: <https://fsharp.github.io/fsharp-core-docs/reference/fsharp-collections-listmodule.html>

FSharp.Core Set Module: <https://fsharp.github.io/fsharp-core-docs/reference/fsharp-collections-setmodule.html>

FSharp.Core Map Module: <https://fsharp.github.io/fsharp-core-docs/reference/fsharp-collections-mapmodule.html>

FSharp.Core Seq Module: <https://fsharp.github.io/fsharp-core-docs/reference/fsharp-collections-seqmodule.html>

Microsoft Learn F# collection types: <https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/fsharp-collection-types>